

Software Design Document

Software Specification Document

System Overview

System Name: Investment Monitoring Financial System

Version: 1.0

Prepared By: Devbot

Date: April 27, 2024

- - -

Purpose

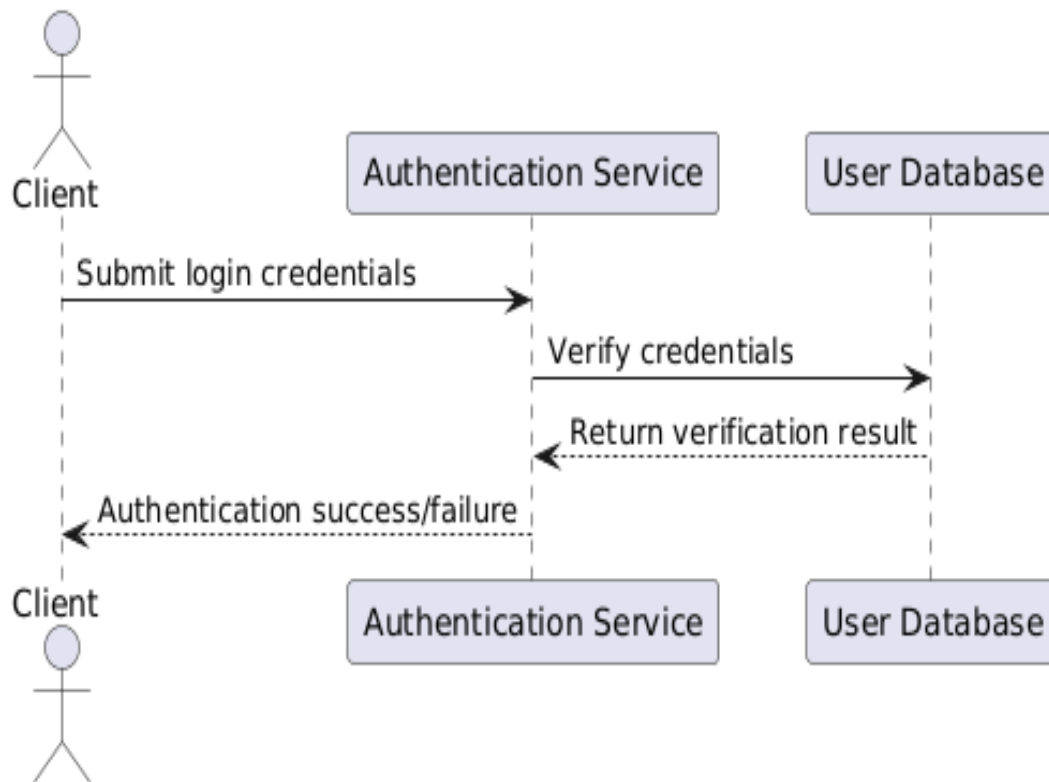
The Investment Monitoring Financial System is designed to provide clients with real-time monitoring and management of their investment portfolios. The system ensures 100% availability to meet the critical nature of financial operations. It prioritizes client data privacy and integrity, ensuring that sensitive financial information is securely handled and maintained. By offering comprehensive investment tracking, reporting, and management tools, the system aims to empower clients to make informed financial decisions while maintaining trust through robust security and reliability measures.

- - -

Sequence Diagrams

Use Case UC-001: User Authentication

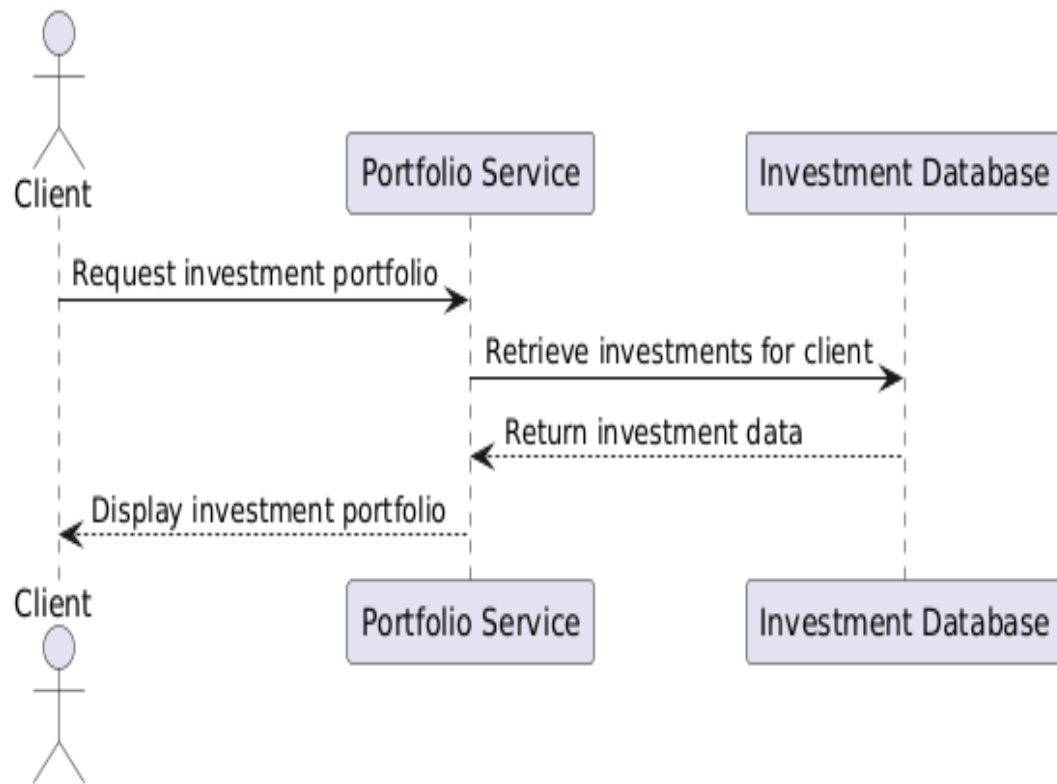
```plantuml



...

#### Use Case UC-002: View Investment Portfolio

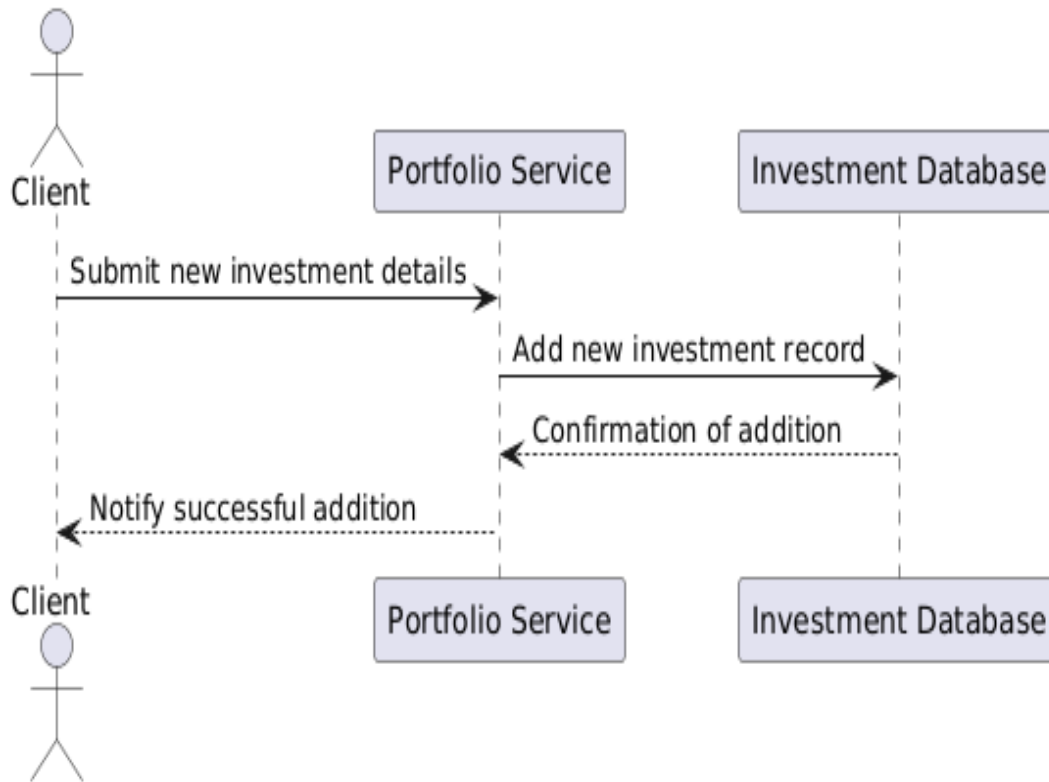
```plantuml



...

Use Case UC-003: Add New Investment

```plantuml



...

\*Note: Additional use cases should follow the same structure with unique identifiers.\*

- - -

## Major Design Decisions

### Significant Design Choices

1. **Microservices Architecture:** The system is decomposed into distinct services (e.g., Authentication, Portfolio Management, Reporting) to enhance scalability, maintainability, and enable independent deployment cycles.
2. **RESTful APIs:** All inter-service communications are handled via RESTful APIs, ensuring interoperability and ease of integration with third-party services or client applications.
3. **Cloud-Based Infrastructure:** Utilizing cloud services ensures high availability, fault tolerance, and scalability to meet the 100% availability requirement.

### Modularization Criteria

The system is modularized based on functional domains:

- **Authentication Module:** Handles all user authentication and authorization processes.
- **Portfolio Management Module:** Manages investment data, including CRUD operations.
- **Reporting Module:** Generates investment reports and analytics.
- **Notification Module:** Manages alerts and notifications to clients.

### Cohesion and Coupling

- **High Cohesion:** Each module has a well-defined purpose, ensuring that related functionalities are encapsulated within the same module.
- **Low Coupling:** Modules interact through well-defined interfaces and APIs, minimizing dependencies and enhancing the flexibility to modify or replace modules without affecting others.

### Non-Functional Requirements

1. **Security:** Implement robust security measures, including encryption of data at rest and in transit, secure authentication mechanisms, and regular security audits to protect client data privacy and integrity.
2. **Reliability:** Achieve system reliability through redundant components, failover strategies, and continuous monitoring to ensure 100% availability and maintain consistent performance.

- - -

## Architecture

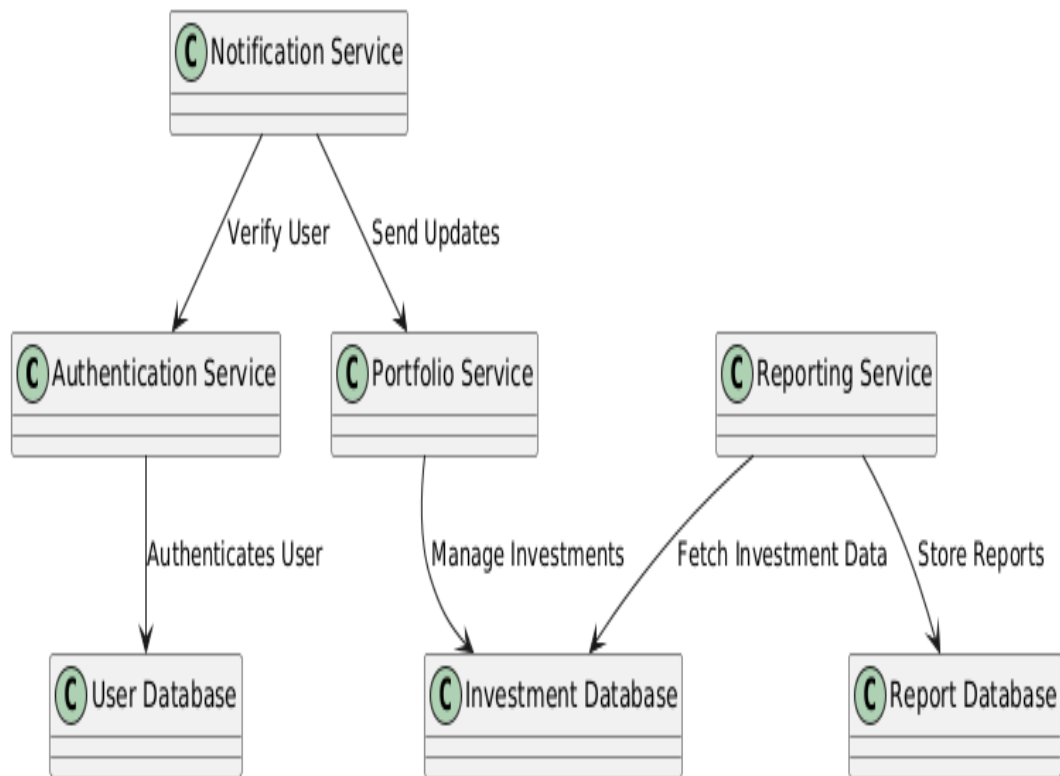
### Architectural Patterns

The system employs a **Microservices Architecture** pattern, which decomposes the application into loosely coupled, independently deployable services. This approach enhances scalability, allows for targeted updates, and isolates failures, contributing to the system's overall reliability and maintainability.

Additionally, the **Event-Driven Architecture** pattern is utilized to handle asynchronous communication between services. This facilitates real-time data processing and ensures that the system can efficiently manage high volumes of transactions without performance degradation.

### Module Decomposition

``plantuml



...

### Component Explanations

- **Authentication Service:**

- **Functionality:** Handles user login, registration, and authentication processes.

- **Exposed Interfaces:**

- `authenticateUser(credentials)`: Validates user credentials.

- `registerUser(userDetails)`: Registers a new user.

- **Portfolio Service:**

- **Functionality:** Manages client investment portfolios, including adding, updating, and retrieving investment data.

- **Exposed Interfaces:**

- `getPortfolio(userId)` : Retrieves the investment portfolio for a specific user.
- `addInvestment(userId, investmentDetails)` : Adds a new investment to a user's portfolio.
- `updateInvestment(userId, investmentId, updates)` : Updates existing investment details.
- `deleteInvestment(userId, investmentId)` : Removes an investment from the portfolio.
- **Reporting Service:**
- **Functionality:** Generates and manages investment reports and analytics.
- **Exposed Interfaces:**
- `generateReport(userId, reportType)` : Creates a report based on specified parameters.
- `getReport(reportId)` : Retrieves a specific report.
- **Notification Service:**
- **Functionality:** Manages system notifications and alerts to clients regarding their investments.
- **Exposed Interfaces:**
- `sendNotification(userId, message)` : Sends a notification to a user.
- `scheduleNotification(userId, message, time)` : Schedules a future notification.
- **Databases:**
- **User Database:** Stores user credentials and profile information.
- **Investment Database:** Stores detailed investment data for each client.
- **Report Database:** Archives generated reports for future reference.
- --

## Use of Design Patterns

1. **Singleton Pattern (Authentication Service):**

- **Rationale:** Ensures that only one instance of the Authentication Service exists, maintaining a consistent state for user sessions and authentication tokens.

## 2. **Factory Pattern (Investment Creation):**

- **Rationale:** Simplifies the creation of various investment types by encapsulating the instantiation logic, promoting scalability and ease of adding new investment types.

## 3. **Observer Pattern (Notification Service):**

- **Rationale:** Enables the Notification Service to subscribe to events (e.g., portfolio changes) and react accordingly, facilitating real-time updates to clients.

## 4. **Repository Pattern (Data Access):**

- **Rationale:** Abstracts the data layer, providing a clean separation between business logic and data access, enhancing testability and maintainability.

## 5. **Adapter Pattern (External Integrations):**

- **Rationale:** Ensures compatibility with third-party services (e.g., financial data providers) by allowing the system to interface with external APIs seamlessly.

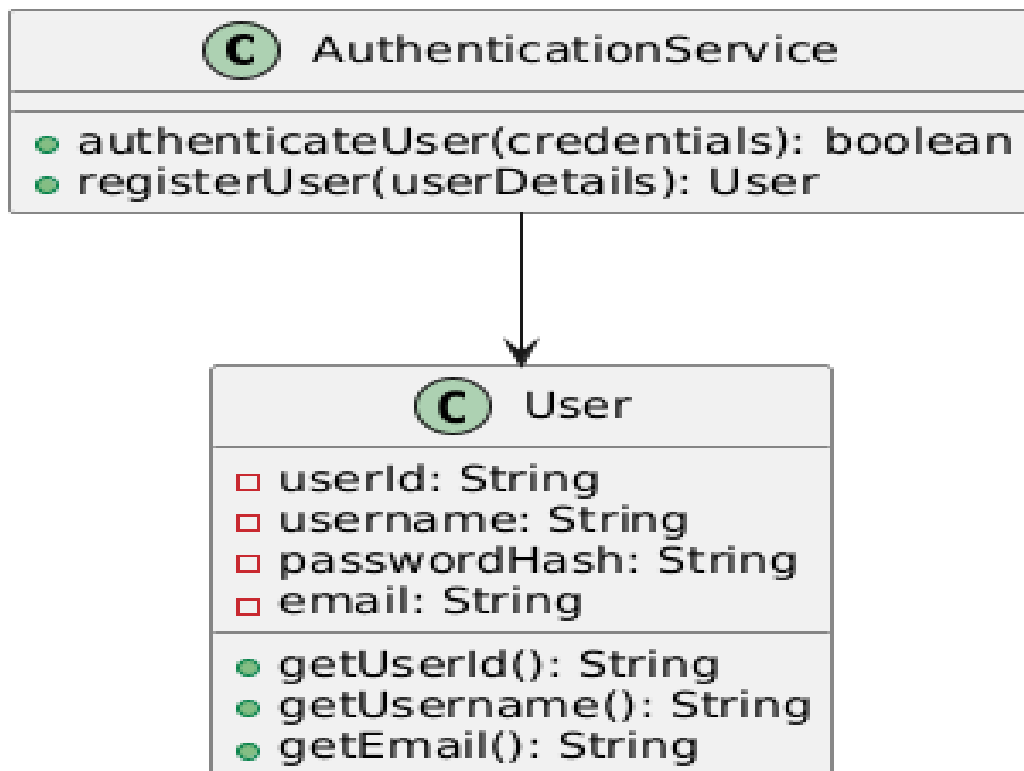
- --

## Detailed Class Diagrams

### Authentication Service

```
``plantuml
```

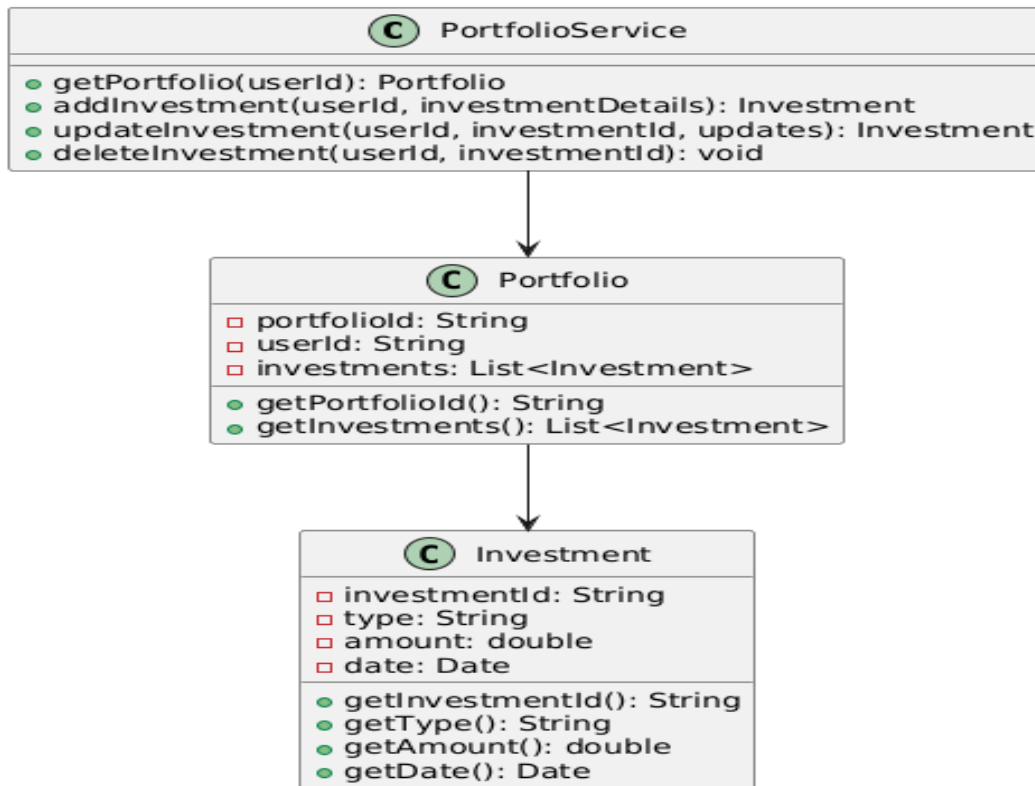




...

**Portfolio Service**

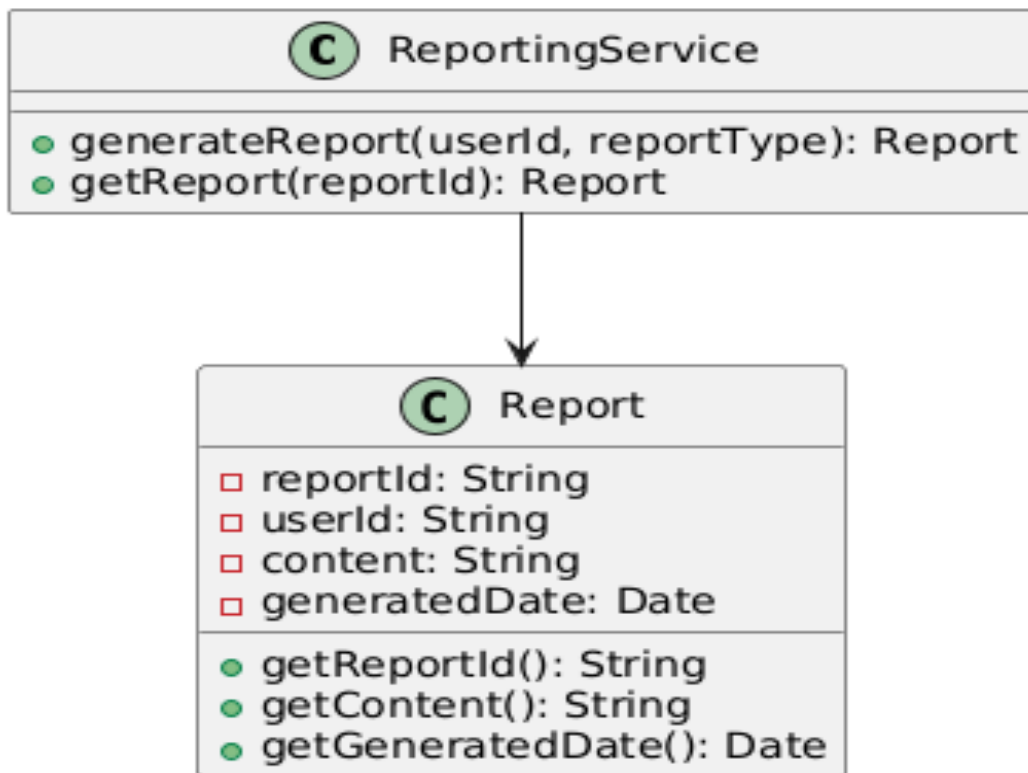
```plantuml



...

Reporting Service

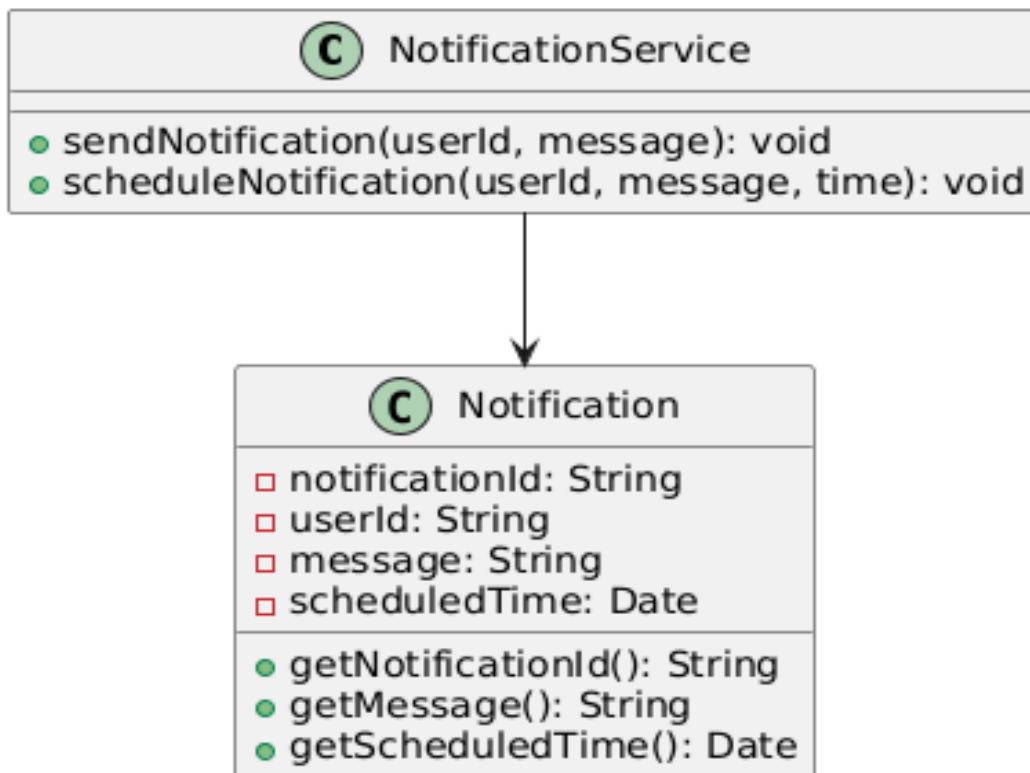
```plantuml



...

**Notification Service**

```plantuml



...

Note: Additional class diagrams should be created for other components as needed.

- - -

Test Driven Development

Test Case 1

- **Test ID:** TC-UC001-01

- **Category:** User Authentication

- **Initial Condition:** The system is running, and the User Database contains a registered user with username "john_doe" and password "SecurePass123".

- **Procedure:**

1. Navigate to the login page.

2. Enter username "john_doe".

3. Enter password "SecurePass123".

4. Click the "Login" button.

- **Expected Outcome:** The system authenticates the user and redirects to the dashboard with a success message.

Test Case 2

- **Test ID:** TC-UC002-01

- **Category:** View Investment Portfolio

- **Initial Condition:** The user "john_doe" is authenticated and has three investments in the Portfolio Database.

- **Procedure:**

1. Navigate to the "My Investments" section.

2. Click on "View Portfolio".

- **Expected Outcome:** The system displays a list of the three investments associated with "john_doe".

Test Case 3

- **Test ID:** TC-UC003-01

- **Category:** Add New Investment

- **Initial Condition:** The user "john_doe" is authenticated.

- **Procedure:**

1. Navigate to the "Add Investment" page.

2. Enter investment details: Type = "Stock", Amount = "10000", Date = "2024-04-25".

3. Click the "Submit" button.

- **Expected Outcome:** The system adds the new investment to "john_doe's" portfolio and displays a confirmation message.

Test Case 4

- **Test ID:** TC-UC001-02

- **Category:** User Authentication

- **Initial Condition:** The system is running, and the User Database does not contain a user with username "jane_doe".

- **Procedure:**

1. Navigate to the login page.
2. Enter username "jane_doe".
3. Enter password "AnyPassword".
4. Click the "Login" button.

- **Expected Outcome:** The system displays an authentication failure message indicating invalid credentials.

Test Case 5

- **Test ID:** TC-UC002-02

- **Category:** View Investment Portfolio

- **Initial Condition:** The user "john_doe" is authenticated but has no investments in the Portfolio Database.

- **Procedure:**

1. Navigate to the "My Investments" section.
2. Click on "View Portfolio".

- **Expected Outcome:** The system displays a message indicating that there are no investments in the portfolio.

Note: Additional test cases should be developed for other use cases and edge scenarios.

- - -

Conclusion

This software specification document outlines the foundational architecture and design principles for the Investment Monitoring Financial System. By adhering to modular design, leveraging proven architectural and design patterns, and emphasizing non-functional requirements such as security and reliability, the system is poised to deliver a robust, scalable, and secure platform for managing and monitoring client investments.